

MECHATRONICS AND IOT

ASSIGNMENT - 15

IoT-Based HVAC Temperature and Humidity Monitoring System

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

IoT-Based HVAC Temperature and Humidity Monitoring System

1. Project Overview

The IoT-Based HVAC Temperature and Humidity Monitoring System is a smart monitoring system designed to continuously measure and monitor the temperature and humidity conditions inside rooms, laboratories, offices, industries, and other enclosed environments. However, continuous monitoring is important to ensure that the temperature and humidity remain within the desired range and that abnormal conditions can be identified quickly.

The proposed system uses an ESP8266 NodeMCU as the main control unit and a DHT22 temperature and humidity sensor for environmental measurement. The DHT22 provides digital readings of both temperature and relative humidity with good accuracy. The measured values can be displayed locally using an OLED/LCD display, allowing users to easily observe the current temperature and humidity.

A relay module is incorporated into the system to control an HVAC-related load such as a cooling fan or other suitable electrical device. When the temperature exceeds the specified threshold, the ESP8266 can activate the relay to switch ON the connected cooling device. This provides an automatic control mechanism and reduces the need for continuous manual monitoring.

Main components

- ESP8266 NodeMCU
- DHT22 Sensor
- Relay Module
- OLED/LCD Display
- Buzzer
- Breadboard

- Jumper Wires

2. Project Statement

To design and develop an IoT-Based HVAC Temperature and Humidity Monitoring System using ESP8266 NodeMCU and DHT22 sensor to continuously monitor environmental conditions, display real-time temperature and humidity values, provide alerts during abnormal conditions, and automatically control HVAC equipment through a relay for improved comfort, safety, and energy efficiency.

The project aims to develop a smart HVAC monitoring system that can continuously sense and analyze temperature and humidity in an indoor environment. The ESP8266 NodeMCU collects data from the DHT22 sensor and displays the measured values on an OLED/LCD display. When the temperature or humidity exceeds the predefined limit, the system activates the buzzer to provide an alert and operates the relay to control the connected HVAC device. The IoT capability of the ESP8266 also makes the system suitable for future remote monitoring and automated HVAC management, helping to improve energy efficiency, environmental comfort, and system reliability.

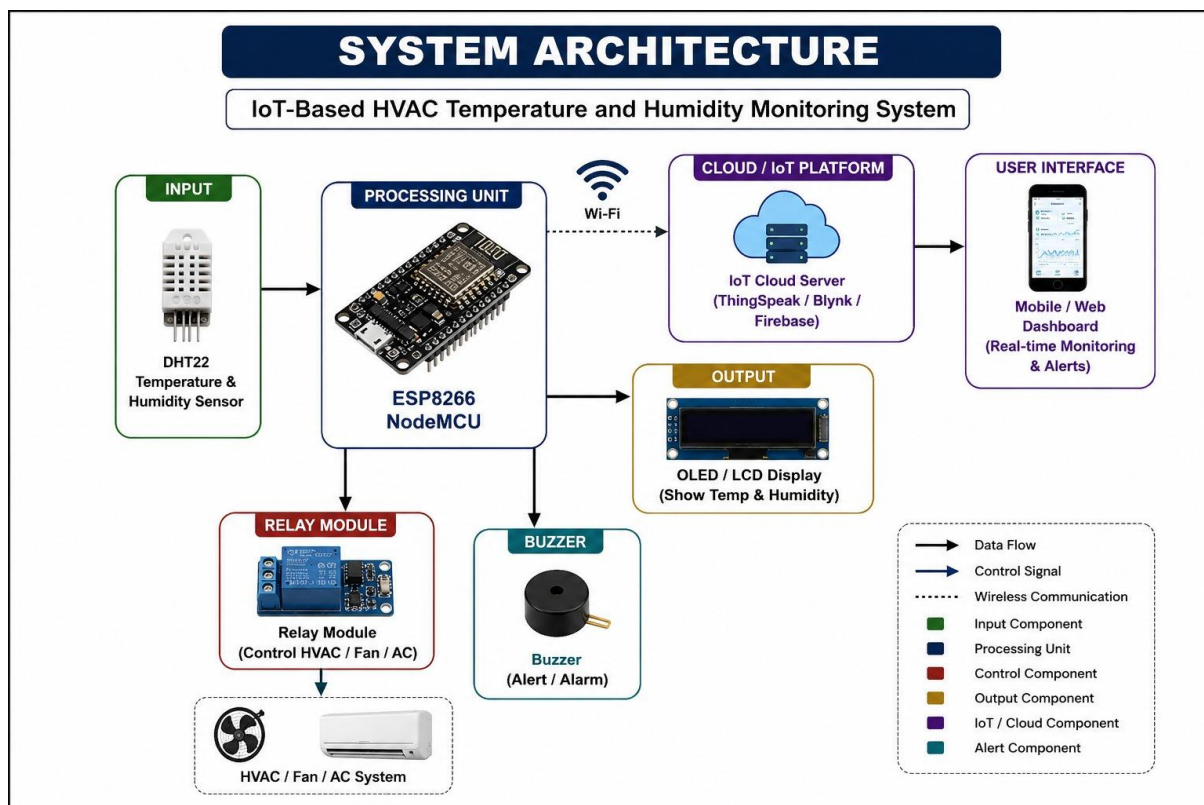
3. Proposed Solution

The proposed solution is to develop a smart HVAC monitoring and control system using an ESP8266 NodeMCU and DHT22 temperature and humidity sensor. The DHT22 continuously measures the environmental temperature and humidity and sends the readings to the NodeMCU for processing. The measured values are displayed on an OLED/LCD display for easy real-time monitoring.

The system also uses a relay module to automatically control a connected HVAC device such as a fan or cooling system based on the measured conditions. The ESP8266 provides Wi-Fi connectivity, allowing the system to be

extended for remote IoT monitoring and control. This solution reduces the need for continuous manual monitoring and helps maintain suitable indoor conditions while improving energy efficiency and user comfort.

4. System Architecture:

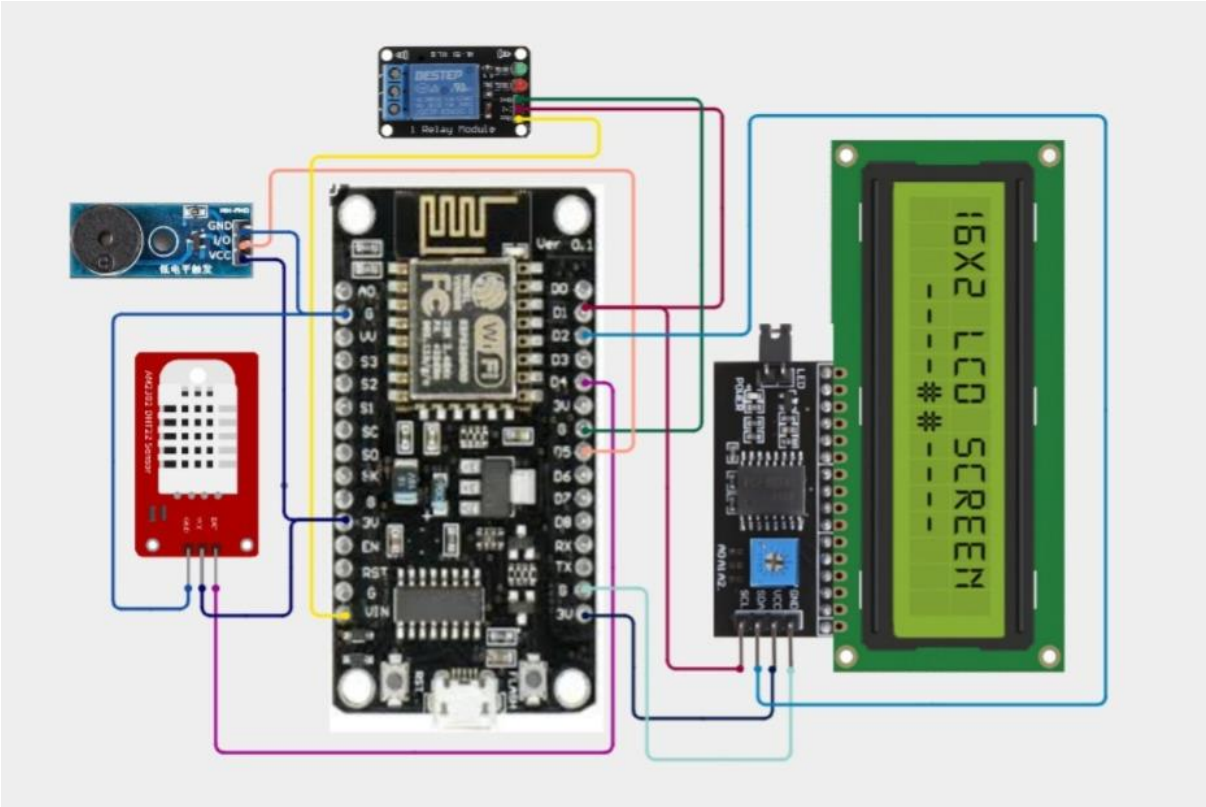


The system consists of an ESP8266 NodeMCU as the main controller. The DHT22 sensor measures temperature and humidity and sends the data to the NodeMCU. The readings are displayed on an OLED/LCD display and can be monitored through the IoT network. Based on predefined temperature or humidity limits, the NodeMCU controls the relay module and activates the buzzer to provide an alert. The system is powered through the NodeMCU and interconnected using jumper wires on a breadboard.

5. Circuit Table:

S.No	Component	Module / Model	Quantity	Connections (From)	Connections (To)	Function	Power Supply
1.	ESP8266 NodeMCU	NodeMCU ESP-12E	1	-	All Modules	Microcontroller with Wi-Fi for data processing and IoT communication	5V (USB)
2.	DHT22 Sensor	DHT22 (AM2302)	1	VCC DATA GND	3V3 (NodeMCU) D4 (GPIO2) GND	Measures Temperature and Humidity	3.3V
3.	Relay Module (1 Channel)	1 Channel 5V Relay Module	1	VCC GND IN	5V (NodeMCU) GND D1 (GPIO5)	Controls HVAC (Fan / AC / Relay Load)	5V
4.	OLED Display (I2C)	0.96" OLED (SSD1306)	1	VCC GND SCL SDA	3V3 (NodeMCU) GND D5 (GPIO14) D6 (GPIO12)	Displays Temperature & Humidity values	3.3V
5.	Buzzer (Active)	Active Buzzer 5V	1	+ -	D3 (GPIO0) GND	Alarm / Alert indication	5V
6.	Breadboard	-	1	-	-	For making connections	-
7.	Jumper Wires	Male-Male / Male-Female	As required	-	-	Electrical Connections	-

6. Circuit Design:



7. Algorithm:

1. Start the System

Switch on the power supply and initialize the ESP8266 NodeMCU.

2. Initialize Components

Configure the DHT22 sensor, OLED/LCD display, relay module, and buzzer. Establish the Wi-Fi connection for IoT monitoring.

3. Read Temperature and Humidity

The DHT22 sensor continuously measures the surrounding temperature (°C) and relative humidity (%).

4. Send Data to NodeMCU

The sensor readings are transmitted to the ESP8266 NodeMCU for processing.

5. Display the Readings

The NodeMCU displays the current temperature and humidity values on the OLED/LCD display.

6. Check Temperature Condition

Compare the measured temperature with the predefined HVAC temperature limits.

7. If the temperature is higher than the set limit, the system activates the relay to turn ON the cooling/HVAC unit.

8. If the temperature is within the acceptable range, the HVAC unit remains in its normal state.

9. If the temperature falls below the required minimum, the HVAC control can be switched OFF or adjusted according to the programmed conditions.

10. Check Humidity Condition

Compare the measured humidity with the predefined humidity limits.

11. If the humidity is above the specified limit, the system identifies an abnormal humidity condition.

12. If the humidity is within the desired range, the system continues normal operation.

13. **Activate Warning System**

If the temperature or humidity exceeds the predefined safe range, the buzzer is activated to provide an alert. This helps the user identify abnormal HVAC conditions quickly.

14. **IoT Monitoring**

The ESP8266 can transmit the temperature and humidity readings through Wi-Fi to an IoT/cloud platform, allowing the environmental conditions to be monitored remotely.

15. **Control HVAC System**

Based on the sensor readings and programmed threshold values, the relay controls the HVAC equipment automatically. This reduces unnecessary operation and helps maintain the desired indoor environment.

16. **Repeat Monitoring**

After completing the above operations, the NodeMCU waits for a short interval and reads the DHT22 sensor again. The process continues repeatedly for real-time monitoring and automatic control.

17. **Stop Condition**

The algorithm continues as long as the system is powered ON. When the power is switched OFF, the monitoring and HVAC control process stops.

8. Coding:

```
#include "AdafruitIO_WiFi.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// =====
// WIFI DETAILS
// =====
```

```

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====

// ADAFRUIT IO DETAILS

// =====

#define IO_USERNAME  "Hair_13"
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
// Create Adafruit IO object
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASSWORD
);

// =====

// ADAFRUIT IO FEEDS

// =====

AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");
AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");
AdafruitIO_Feed *hvacFeed =
  io.feed("hvac-status");
AdafruitIO_Feed *alertFeed =
  io.feed("hvac-alert");

// =====

// DHT11

// =====

#define DHT_PIN 4
#define DHT_TYPE DHT11

```

```

DHT dht(
  DHT_PIN,
  DHT_TYPE
);
// =====

// RELAY
// =====

#define RELAY_PIN 26
// Most relay modules are ACTIVE LOW
#define RELAY_ON LOW
#define RELAY_OFF HIGH
// =====

// BUZZER
// ===== #define
BUZZER_PIN 25
// =====

// OLED
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1 );
// =====

// VARIABLES
// =====

float temperature = 0;
float humidity = 0;
bool hvacStatus = false;
bool hvacAlert = false;
// =====

// HVAC SETTINGS
// =====

```

```

// HVAC turns ON at 28°C float temperatureON = 28.0;
// HVAC turns OFF at 25°C
float temperatureOFF = 25.0;
// Humidity alert limit
float humidityLimit = 75.0;
// =====
// TIMING
// =====
unsigned long lastSend = 0;
const unsigned long sendInterval = 15000;
// =====
// SETUP
// =====
void setup()
{
  Serial.begin(9600);
  delay(1000);
  // -----
  // DHT11
  // -----
  dht.begin();
  // -----
  // RELAY
  // -----
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, RELAY_OFF );
  // -----
  // BUZZER
  // -----

  pinMode(BUZZER_PIN, OUTPUT);

```

```

digitalWrite(BUZZER_PIN, LOW );
// -----

// OLED
// -----

Wire.begin(21, 22);

if (!display.begin(SSD1306_SWITCHCAPVCC,0x3C))

{
    Serial.println("OLED not detected!");
}
else
{
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(10, 10);
    display.println( "HVAC MONITOR" );
    display.setCursor(10, 30);
    display.println("Starting...");
    display.display();
}
// -----

// ADAFRUIT IO CONNECTION
// -----

Serial.println();
Serial.println("Connecting to Adafruit IO...");
io.connect();
while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}

```

```

}

Serial.println();
Serial.println("Connected to Adafruit IO!" );
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(0, 10);
    display.println("Adafruit IO");
    display.setCursor(0, 30);
    display.println("Connected!");
    display.display();
    delay(2000);
}

// =====
// LOOP
// =====

void loop()
{
    // Keep Adafruit IO connection alive io.run();
    // -----
    // SEND DATA EVERY 15 SECONDS
    // -----
    if (millis() - lastSend >= sendInterval)
    {
        lastSend = millis();

        // =====
        // READ DHT11
        // =====

        temperature =
dht.readTemperature();

        humidity = dht.readHumidity();

```

```

// -----
// CHECK SENSOR
// -----
if ( isnan(temperature) || isnan(humidity))
{
    Serial.println("ERROR: DHT11 reading failed!" );
    return;
}

// =====
// SERIAL SENSOR OUTPUT
// =====

Serial.println();
Serial.println("===== SENSOR DATA =====");
Serial.print("Temperature: ");
Serial.print( temperature );

Serial.println(" C");
Serial.print("Humidity: ");
Serial.print(humidity);

Serial.println( " %");

// =====
// HVAC CONTROL
// =====

if ( temperature >= temperatureON)  {
    hvacStatus = true;
    digitalWrite(RELAY_PIN,RELAY_ON);
} else if (temperature <= temperatureOFF )  {
    hvacStatus = false;
    digitalWrite(RELAY_PIN,RELAY_OFF);
}

// =====

```

```

// HVAC ALERT
// =====
if (temperature >= temperatureON || humidity >= humidityLimit)  {
    hvacAlert = true;

    digitalWrite(BUZZER_PIN,HIGH);
}
else
{
    hvacAlert = false;
    digitalWrite(BUZZER_PIN, LOW );
}
// =====

// SERIAL HVAC STATUS
// =====

Serial.println();
Serial.println("===== HVAC STATUS =====");
Serial.print("HVAC Status: ");
if (hvacStatus)
{
    Serial.println("ON");
}
else
{
    Serial.println("OFF");
}

Serial.print( "HVAC Status Value Sent: ");
Serial.println( hvacStatus ? 1 : 0);
// =====

// SERIAL ALERT STATUS
// =====

```

```

Serial.println();
Serial.println("===== HVAC ALERT =====");
Serial.print( "HVAC Alert: " );
if (hvacAlert)
{
    Serial.println("ALERT");
}
else
{
    Serial.println("NORMAL");
}
Serial.print( "HVAC Alert Value Sent: " );
Serial.println(hvacAlert ? 1 : 0);
// =====
// SEND TEMPERATURE
// =====

Serial.println();
Serial.println( "===== ADAFRUIT IO =====");
Serial.print "Sending Temperature: " );
Serial.println temperature);
    temperatureFeed->save(temperature);
// =====
// SEND HUMIDITY
// =====
Serial.print( "Sending Humidity: " );
Serial.println( humidity );
humidityFeed->save(humidity);
// =====
// SEND HVAC STATUS
// =====
int hvacValue = hvacStatus ? 1 : 0;

```

```

    Serial.print("Sending HVAC Status: " );
    Serial.println(hvacValue);
    hvacFeed->save(hvacValue);
    // =====
    // SEND HVAC ALERT
    // =====
    int alertValue = hvacAlert ? 1 : 0;
    Serial.print( "Sending HVAC Alert: " );
    Serial.println( alertValue );
    alertFeed->save( alertValue);
    // Give Adafruit IO time to process queued
    messagesio.run();
    Serial.println();
    Serial.println( "All data sent to Adafruit IO." );
    Serial.println( "=====");
    // =====
    // OLED DISPLAY
    // =====
    display.clearDisplay();
    display.setTextColor( SSD1306_WHITE );
    display.setTextSize(1);

    // Temperature
    display.setCursor(0, 0); display.print("T:");

    display.print( temperature,);
    display.print("C ");

    // Humidity
    display.print("H:");

    display.print(humidity,0);
    display.println("%");

```

```

    // HVAC Status
display.setCursor(0, 20);
    display.print("HVAC: ");
    if (hvacStatus)
    {
        display.println("ON");
    }
else
    {
        display.println("OFF");
    }

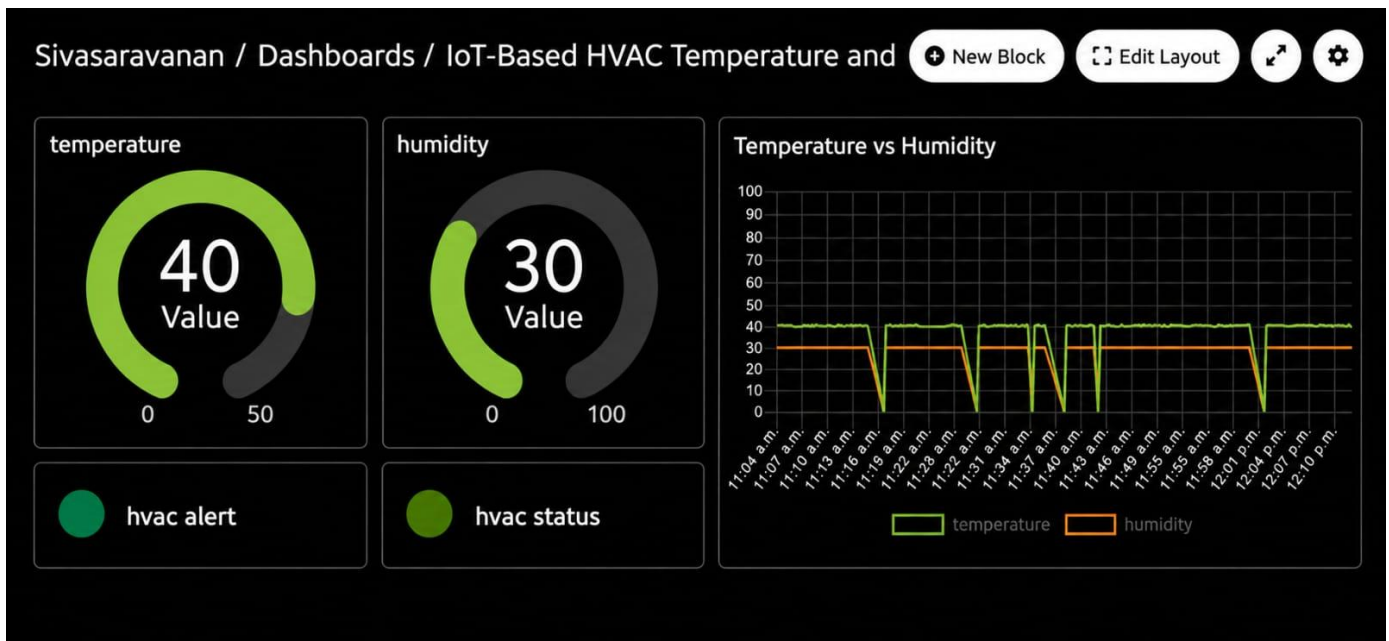
    // Alert
display.setCursor(0, 35);
    display.print("Alert: ");
    if (hvacAlert)
    {
        display.println("ALERT!");
    }
else
{
    display.println("Normal");
}

    // IoT
display.setCursor(0, 50);
    if (io.status() >= AIO_CONNECTED) {
        display.println("IoT: Connected"); } else {
        display.println("IoT: Offline");
    }
    display.display();

    Serial.println "===== ";
}}

```

9. System Working :



The IoT-Based HVAC Temperature and Humidity Monitoring System works by continuously measuring the surrounding temperature and humidity using the DHT22 sensor. The sensor sends the collected data to the ESP8266 NodeMCU, which acts as the main processing and control unit. The measured values are displayed on the OLED/LCD display for easy observation.

The NodeMCU compares the sensor readings with predefined temperature and humidity threshold values. When the temperature exceeds the desired limit, the relay module is activated to control the HVAC cooling system. If the measured values return to the normal range, the relay can be switched OFF according to the programmed conditions. When an abnormal temperature or humidity condition is detected, the buzzer provides an alert to the user.

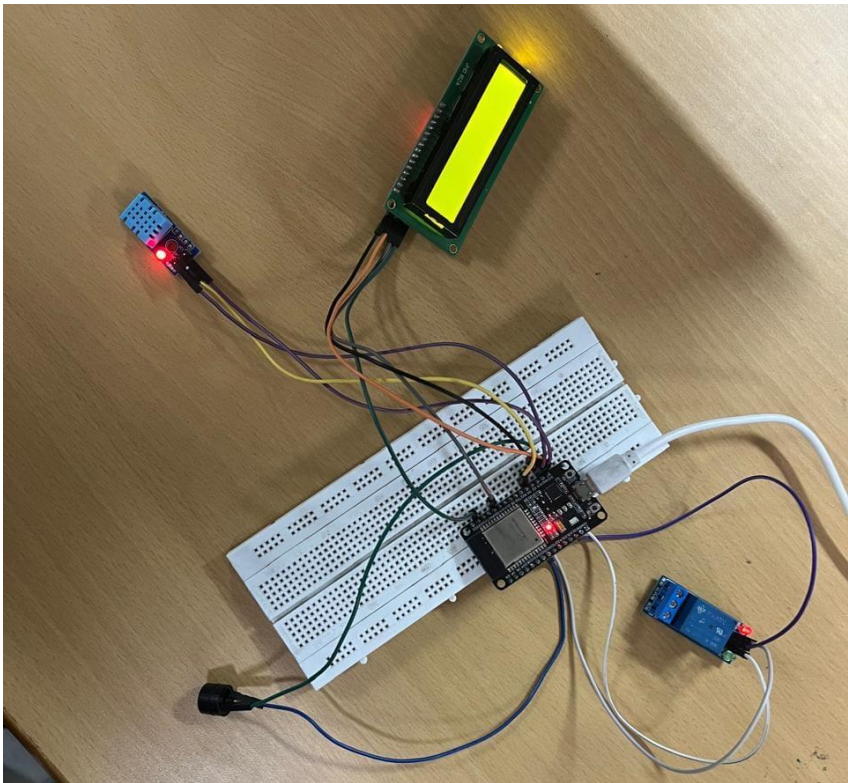
The ESP8266 uses its built-in Wi-Fi capability to transmit the sensor data to an IoT platform, allowing the environmental conditions to be monitored remotely. The system continuously repeats the process of sensing,

processing, displaying, controlling, and monitoring, providing real-time HVAC management and helping improve energy efficiency.

10. BILL OF MATERIAL:

S.No	Component	Specification / Description	Quantity	Unit	Approx. Cost (INR)	Image
1.	ESP8266 NodeMCU	ESP-12E based Wi-Fi development board, 3.3V logic	1	Nos	350	
2.	DHT22 Sensor	Digital Temperature and Humidity Sensor (AM2302)	1	Nos	250	
3.	Relay Module (1 Channel)	5V 1 Channel Relay Module with opto-coupler, Low level trigger	1	Nos	120	
4.	OLED Display (0.96 inch I2C)	0.96" OLED Display SSD1306 I2C Interface, 128x64	1	Nos	200	
5.	Buzzer (Active)	Active Buzzer, 5V	1	Nos	30	
6.	Breadboard	Solderless Breadboard (830 points)	1	Nos	100	
7.	Jumper Wires	Male-Male / Male-Female Jumper Wires	As required	Set	80	

11. Prototype Implementation:



The prototype of the IoT-Based HVAC Temperature and Humidity Monitoring System is implemented using an ESP8266 NodeMCU, DHT22 temperature and humidity sensor, relay module, OLED/LCD display, buzzer, breadboard, and jumper wires. The ESP8266 acts as the main controller and connects the sensor, display, relay, and buzzer.

The DHT22 sensor is connected to the NodeMCU to measure the surrounding temperature and humidity. The sensor readings are processed by the NodeMCU and displayed on the OLED/LCD screen. Predefined threshold values are programmed in the controller to identify abnormal environmental conditions.

The relay module is connected to the HVAC control circuit. When the temperature exceeds the specified limit, the NodeMCU activates the relay to turn ON the cooling system. When the temperature returns to the desired range, the relay is switched OFF according to the programmed conditions. The buzzer is used as an alert when the temperature or humidity exceeds the set limits.

The ESP8266's built-in Wi-Fi connectivity enables the sensor data to be transmitted to an IoT platform for remote monitoring. All components are assembled on a breadboard using jumper wires, making the prototype easy to test and modify.

Thus, the prototype demonstrates real-time temperature and humidity monitoring, automatic HVAC control, local display, abnormal-condition alerts, and IoT-based remote monitoring.

12. Results and Performance Analysis:

The IoT-Based HVAC Temperature and Humidity Monitoring System was successfully implemented and tested using the ESP8266 NodeMCU, DHT22 sensor, relay module, OLED/LCD display, and buzzer. The system continuously measured the surrounding temperature and humidity and displayed the readings on the display in real time.

During testing, the DHT22 sensor provided reliable temperature and humidity measurements. The ESP8266 processed the sensor data and compared it with the predefined threshold values. When the temperature exceeded the set limit, the relay was activated to control the HVAC cooling system. The buzzer also provided an alert when abnormal environmental conditions were detected.

The OLED/LCD display provided immediate information about the current temperature and humidity, making it easy for the user to monitor the indoor environment. The Wi-Fi capability of the ESP8266 also enabled the collected data to be transmitted for IoT-based remote monitoring.

Performance Analysis

The prototype showed good performance in terms of:

- Real-time monitoring: Temperature and humidity were continuously monitored.
- Automatic control: The relay responded automatically when the temperature crossed the predefined threshold.
- Alert system: The buzzer provided a warning during abnormal conditions.
- Display performance: Sensor readings were clearly displayed on the OLED/LCD.
- IoT connectivity: The ESP8266 enabled wireless transmission of environmental data.

Overall Result

The prototype demonstrated that the proposed system can provide effective temperature and humidity monitoring with automatic HVAC control and IoT connectivity. The system can be further improved by adding cloud data storage, mobile notifications, multiple sensors, and advanced automatic HVAC control algorithms.